RISC-V

Instruction Set Architecture

Intruzioni di tipo R

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
0000000		rs2	rs1	000	rd	0110011	ADD
0100000		rs2	rs1	000	rd	0110011	SUB
0000000		rs2	rs1	001	rd	0110011	SLL
0000000		rs2	rs1	010	rd	0110011	SLT
0000000		rs2	rs1	011	rd	0110011	SLTU
0000000		rs2	rs1	100	rd	0110011	XOR
0000000		rs2	rs1	101	rd	0110011	SRL
0000000		rs2	rs1	110	rd	0110011	OR
0000000		rs2	rs1	111	rd	0110011	AND

Codifica e decodifica

Codifica

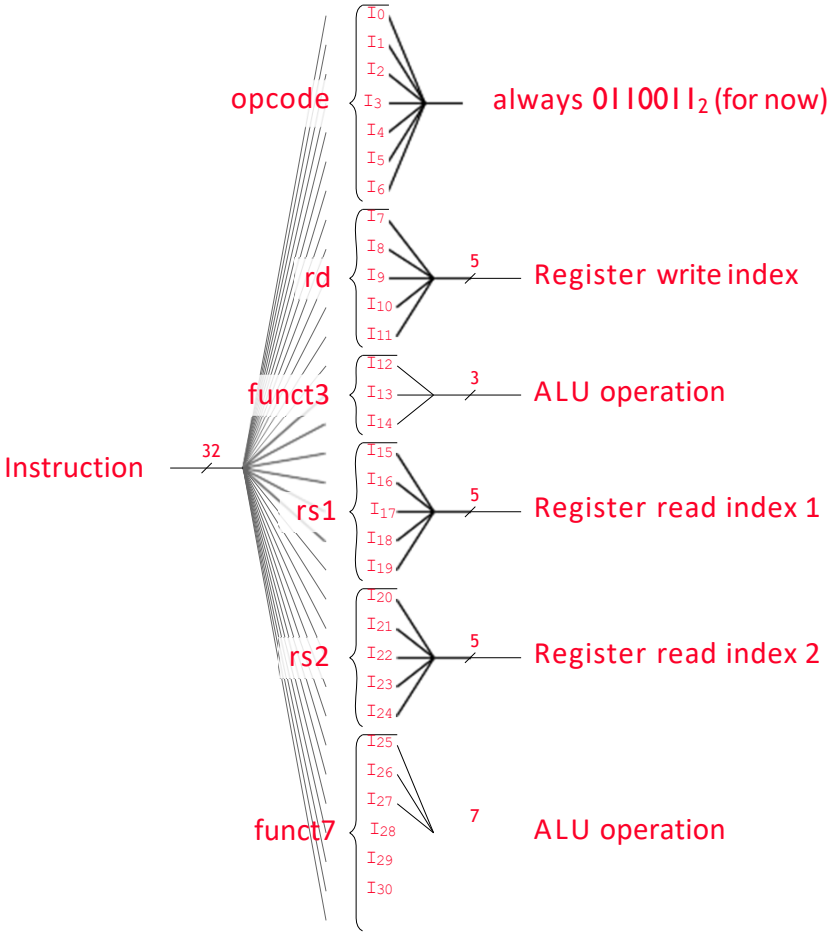
```
xor x8, x4, x5    # 0x00524433 (0000 0000 0101 0010 0100 0100 0011 0011)
sub x6, x15, x1   # 0x40178333 (0100 0000 0001 0111 1000 0011 0011 0011)
```

Decodifica

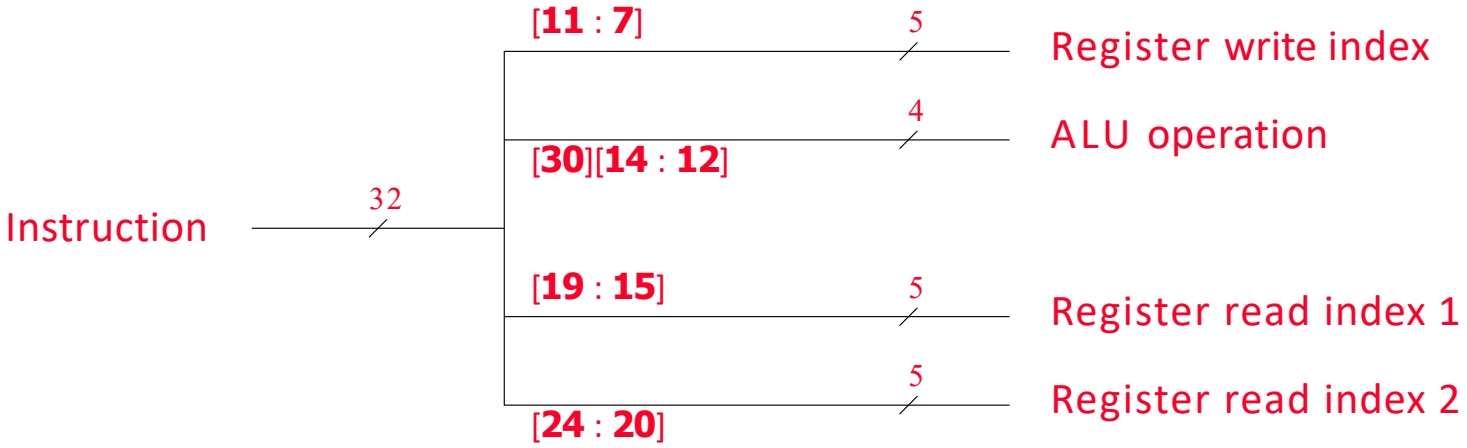
```
0x003110b3 # sll x1, x2, x3

0x007067b3 # or x15, x0, x7
```

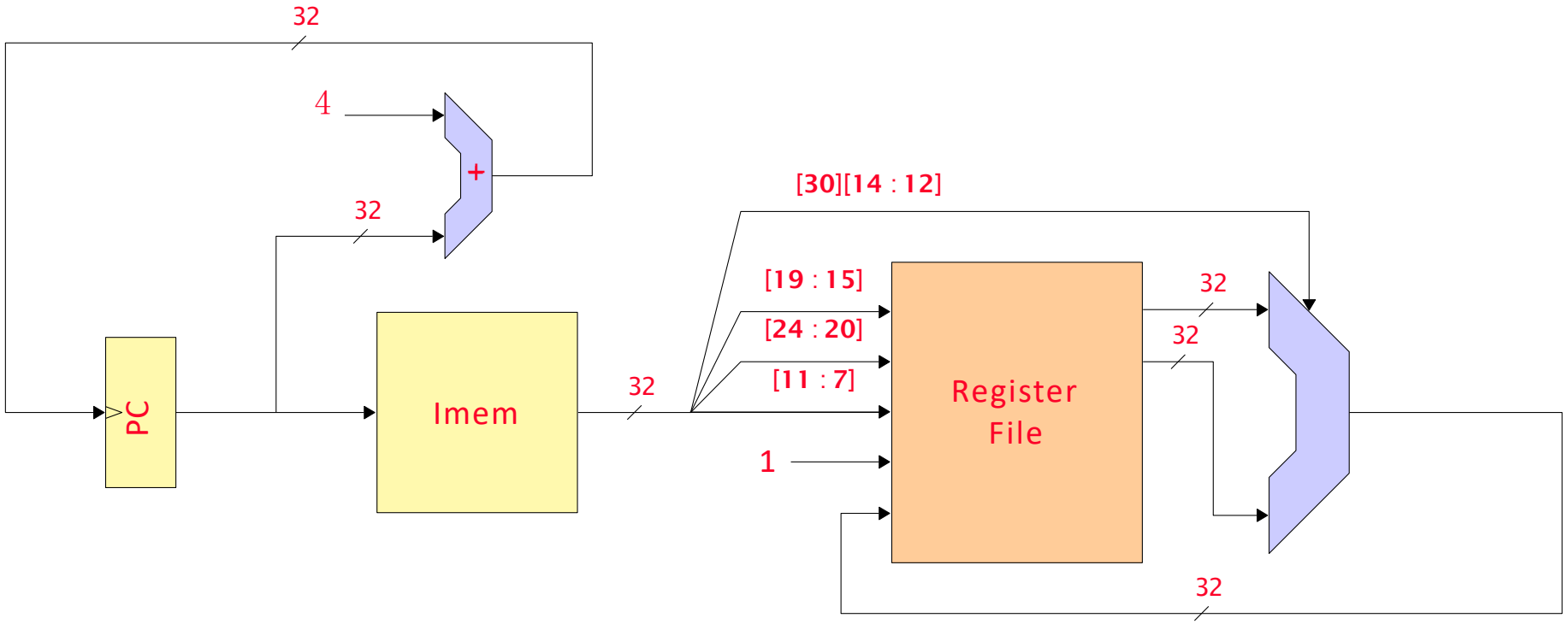
Decodifica istruzioni R-type



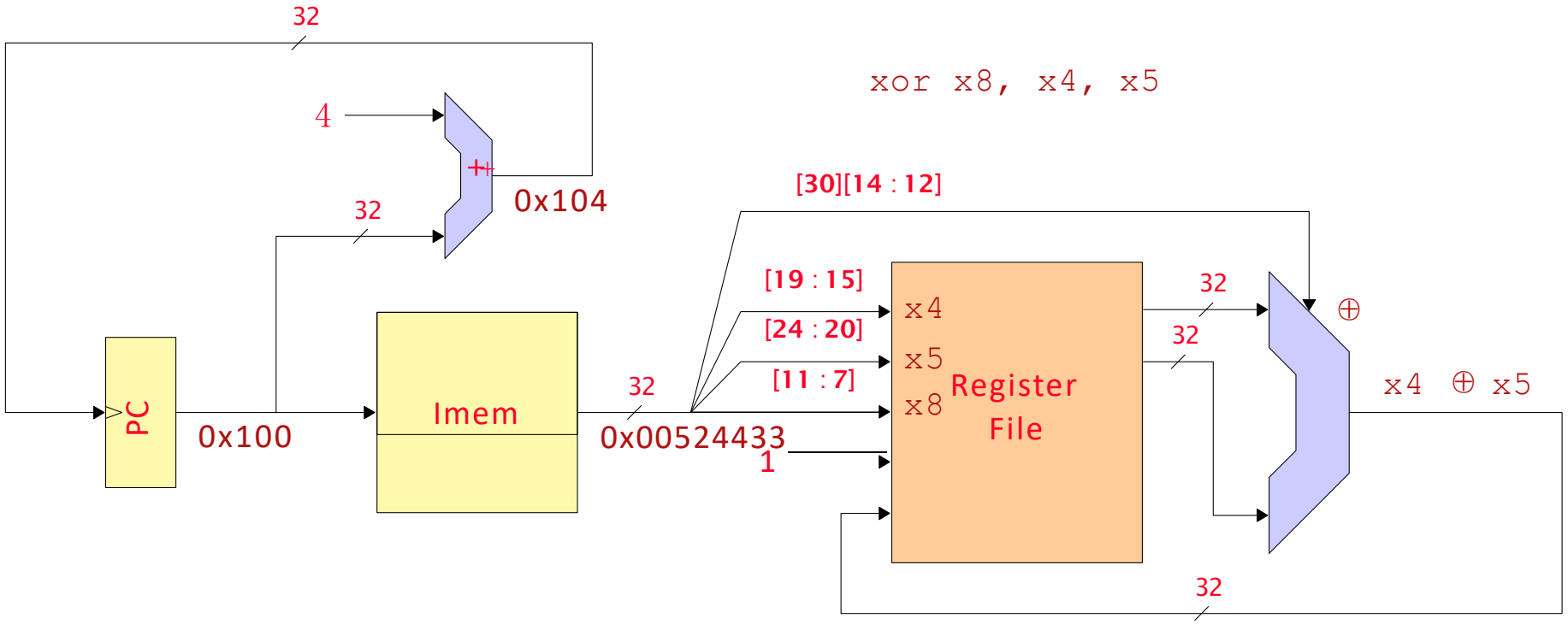
Decodifica istruzioni R-type



Esempio; esecuzione XOR



Esempio; esecuzione XOR

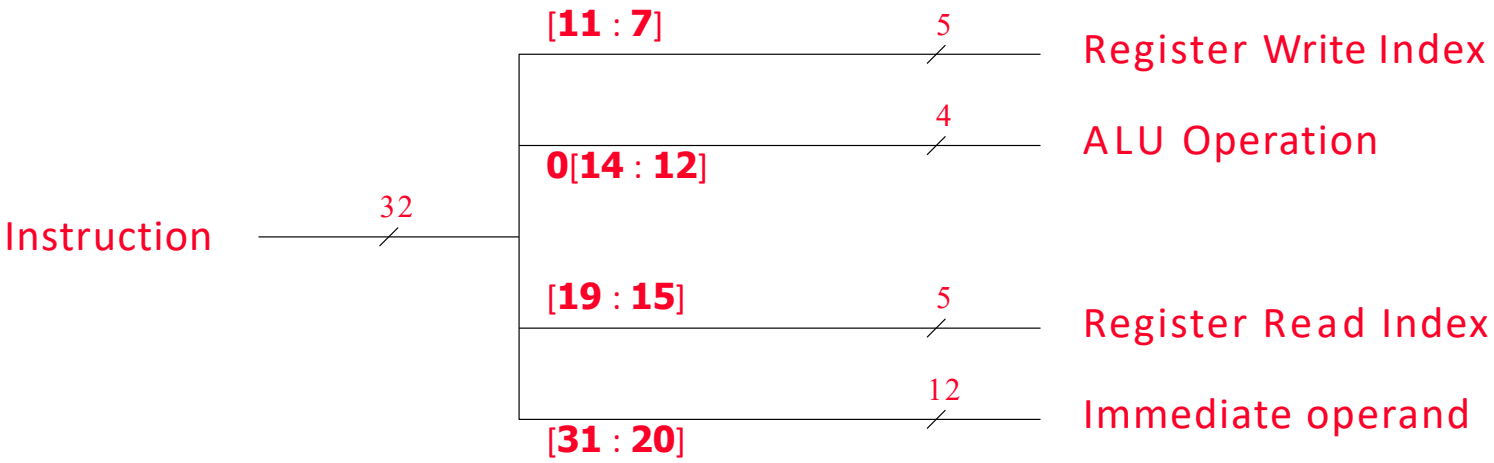


Istruzioni di tipo I

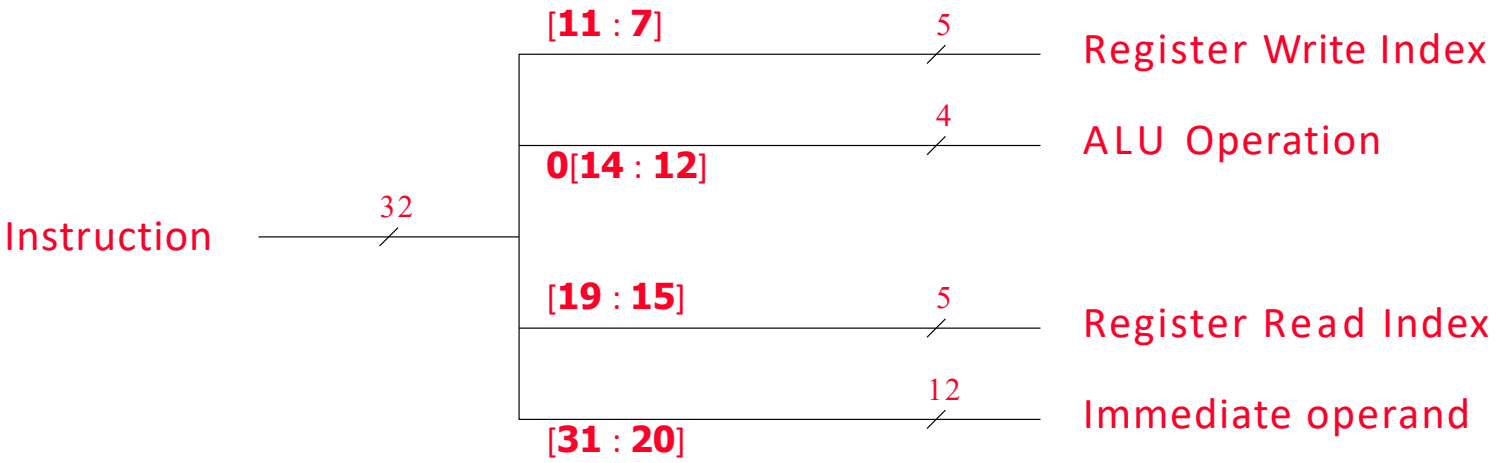
31 27	26 25	24 20	19 15	14 12	11 7	6 0	
imm[11:0]			rs1	funct3	rd	opcode	I-type

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
imm[11:0]			rs1	000	rd	0010011	ADDI
imm[11:0]			rs1	010	rd	0010011	SLTI
imm[11:0]			rs1	011	rd	0010011	SLTIU
imm[11:0]			rs1	100	rd	0010011	XORI
imm[11:0]			rs1	110	rd	0010011	ORI
imm[11:0]			rs1	111	rd	0010011	ANDI
0000000		shamt	rs1	001	rd	0010011	SLLI
0000000		shamt	rs1	101	rd	0010011	SRLI

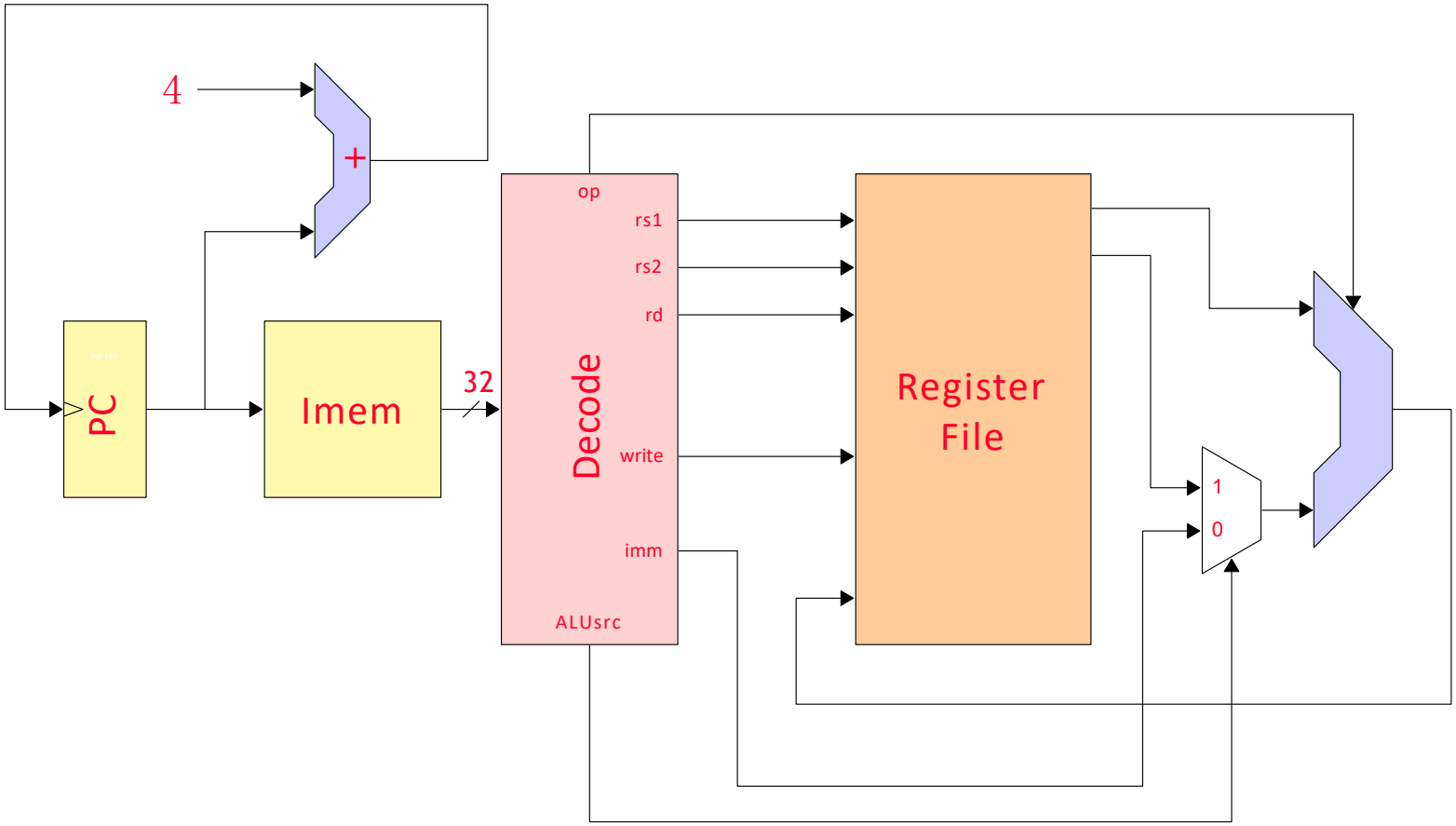
Decodifica


Instruction
32
[11 : 7]
5
Register Write Index
4
ALU Operation
0[14 : 12]
[19 : 15]
5
Register Read Index
12
Immediate operand
[31 : 20]

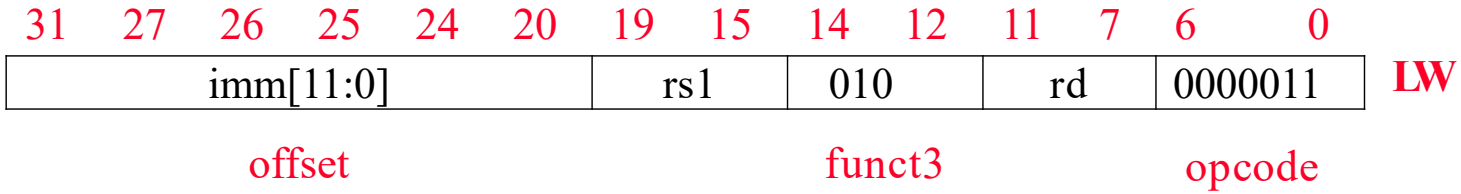
Decodifica


Instruction
32
[11 : 7]
5
Register Write Index
4
ALU Operation
0[14 : 12]
[19 : 15]
5
Register Read Index
12
Immediate operand
[31 : 20]

Data Path

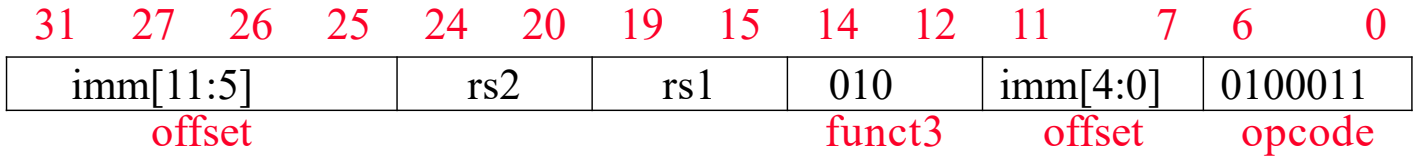

4
+
PC
Imem
32
op
rs1
rs2
rd
Decode
write
imm
ALUsrc
Register
File
1
0

Istruzioni Load/Store



LW

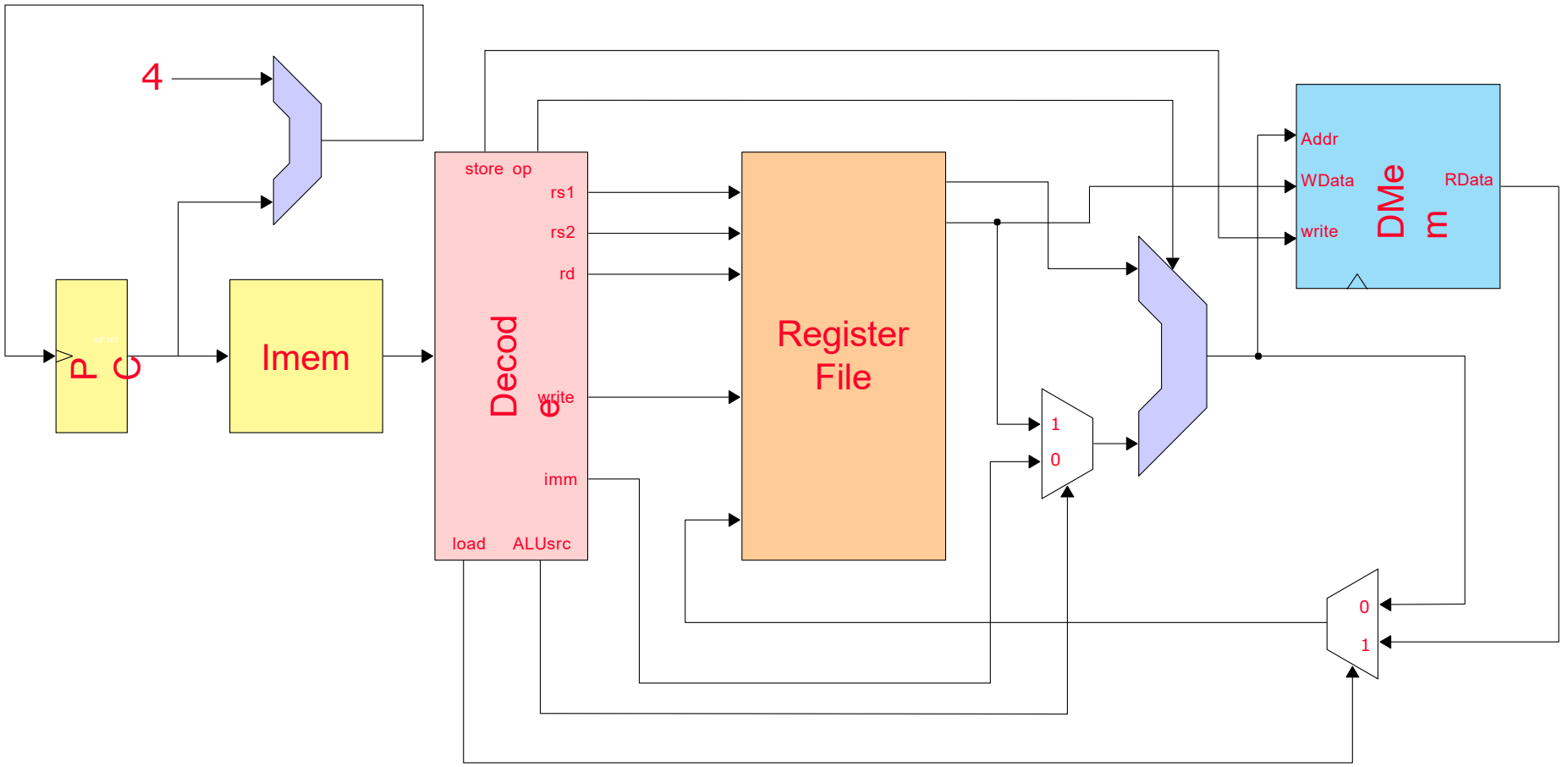
Esempio: `lw s0, 8(a0)` carica MEM$[a0 + 8]$ nel registro *s0*



SW

Esempio : `sw s0, 8(a0)` salva il registro *s0* in MEM$[a0 + 8]$

Data Path


4
PC
Imem
store op
rs1
rs2
rd
write
imm
load
ALUsrc
Decode
Register File
1
0
Addr
WData
write
DMem
RData
0
1

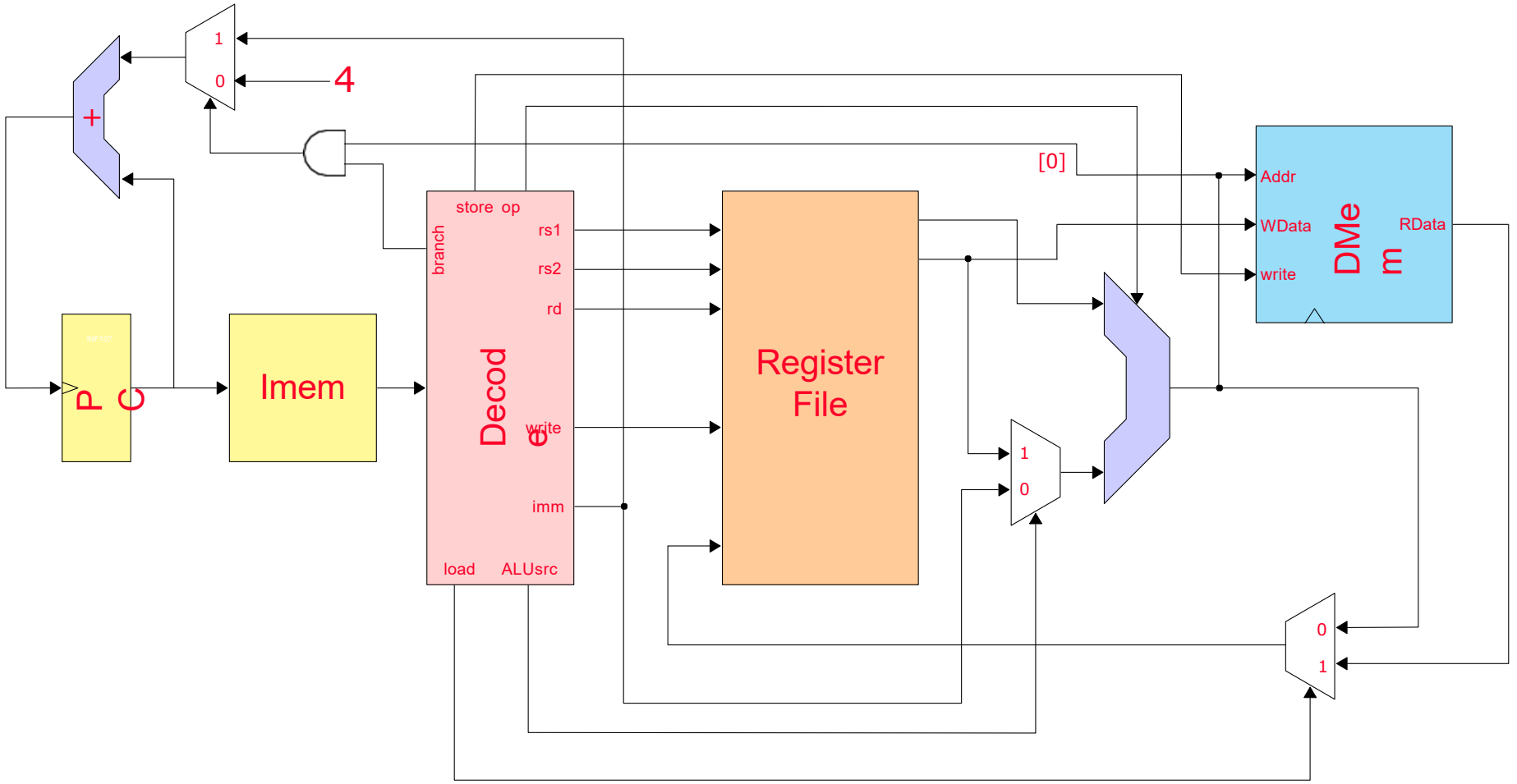
Salti condizionati

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode	B-type

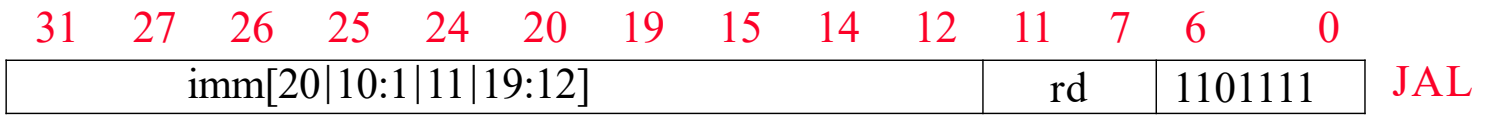
31 27	26 25	24 20	19 15	14 12	11 7	6 0	
imm[12|10:5]		rs2	rs1	000	imm[4:1|11]	1100011	BEQ
imm[12|10:5]		rs2	rs1	001	imm[4:1|11]	1100011	BNE
imm[12|10:5]		rs2	rs1	100	imm[4:1|11]	1100011	BLT
imm[12|10:5]		rs2	rs1	101	imm[4:1|11]	1100011	BGE
imm[12|10:5]		rs2	rs1	110	imm[4:1|11]	1100011	BLTU
imm[12|10:5]		rs2	rs1	111	imm[4:1|11]	1100011	BGEU

- `beq`: Branch if equal
- `bne`: Branch if not equal
- `blt`: Branch if less than
- `bge`: Branch if greater or equal
- `bltu`: Branch if less than unsigned
- `bgeu`: Branch if greater or equal unsigned

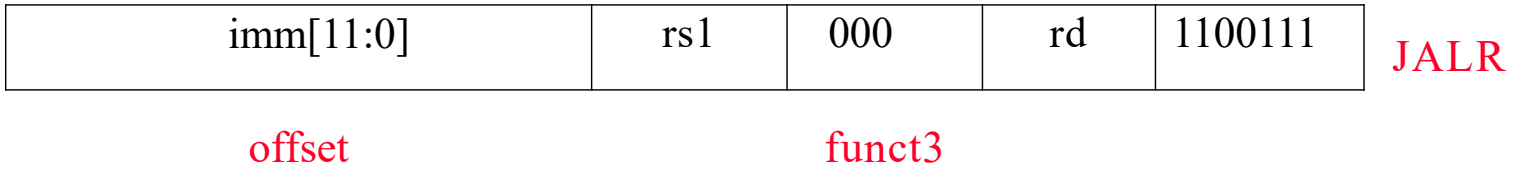
Data Path per istruzioni B-type



Salti incondizionati : Jump/Call

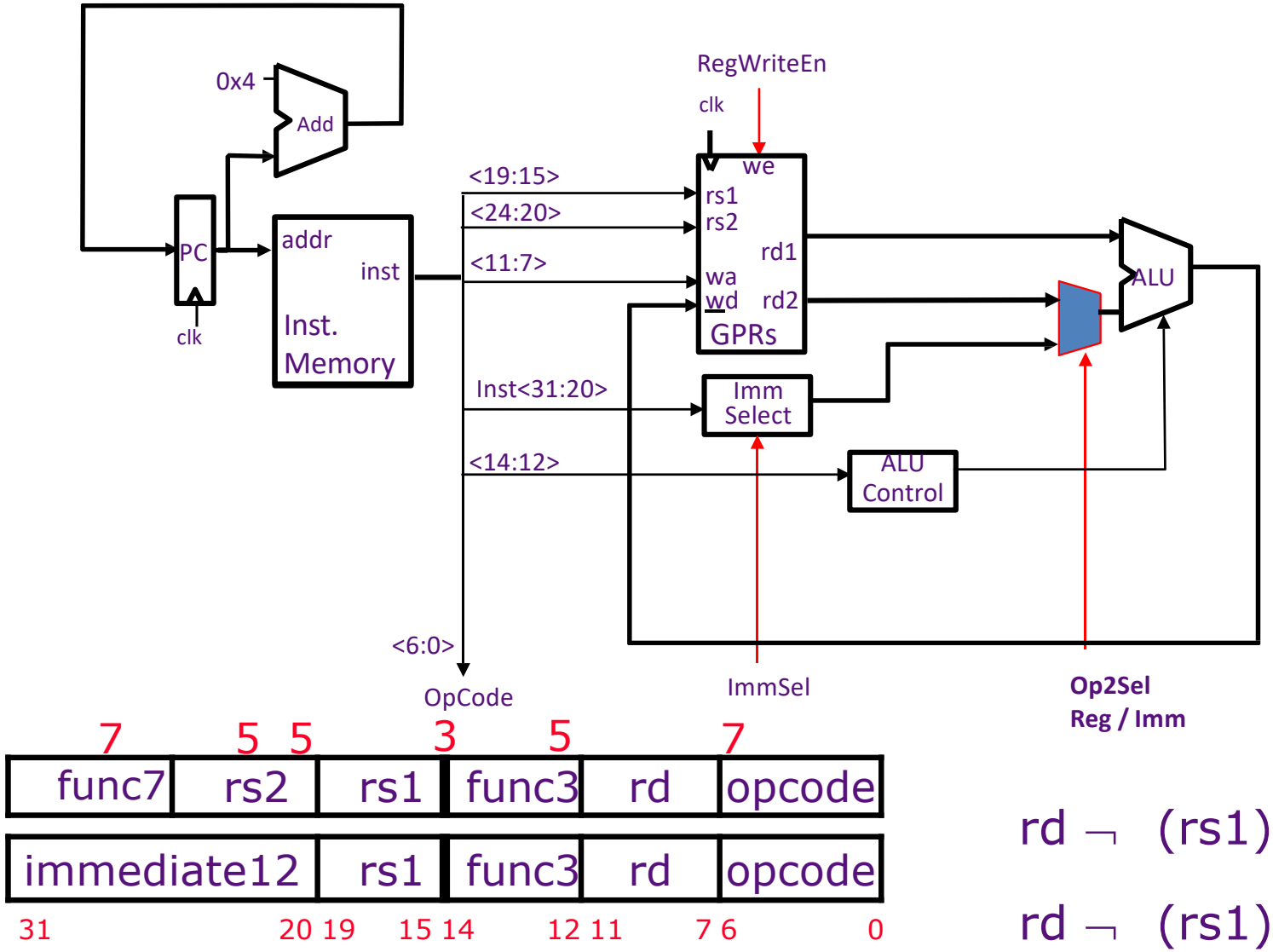


`jal`: *jump and link*



`jalr`: *jump and link register*

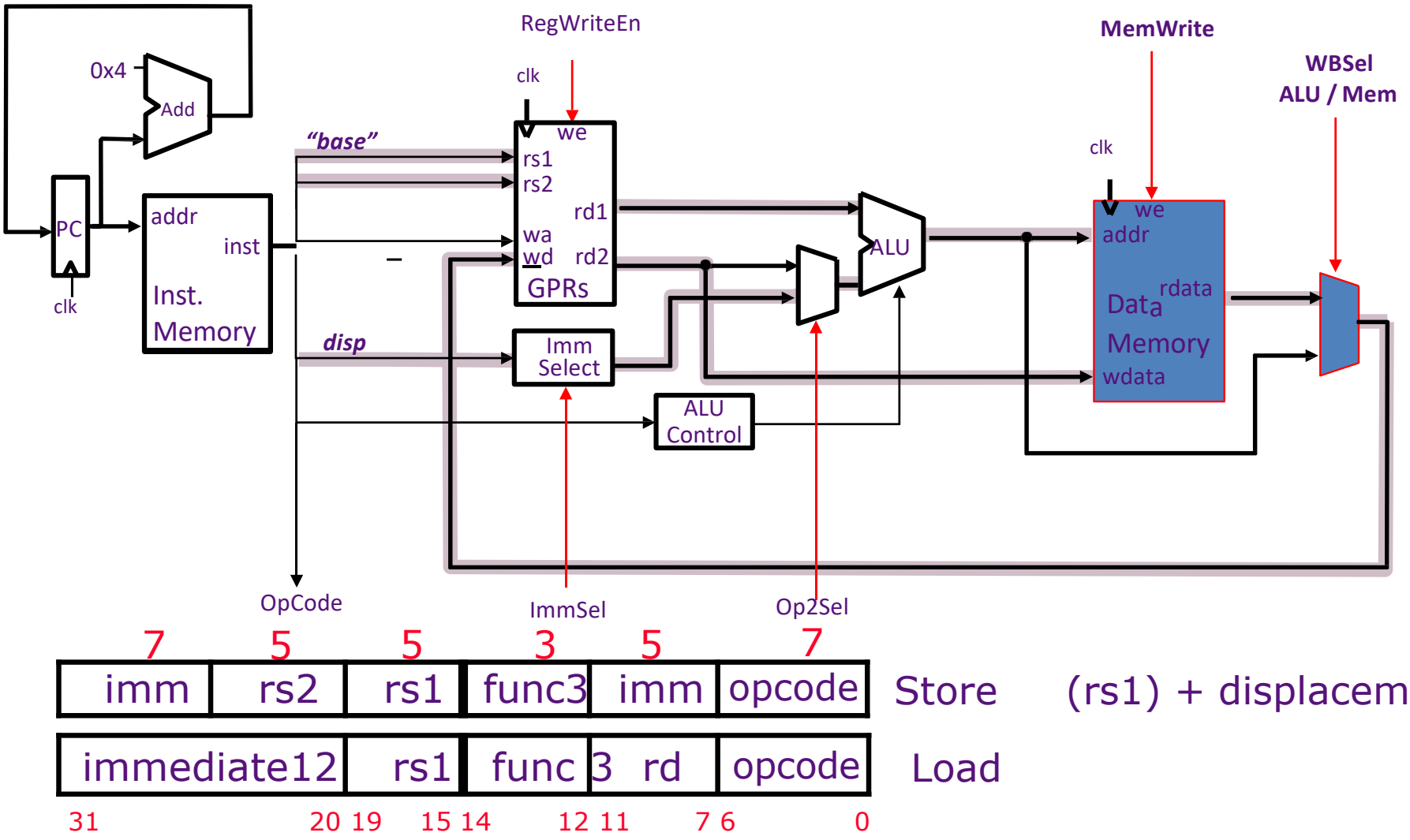
Datapath unificato



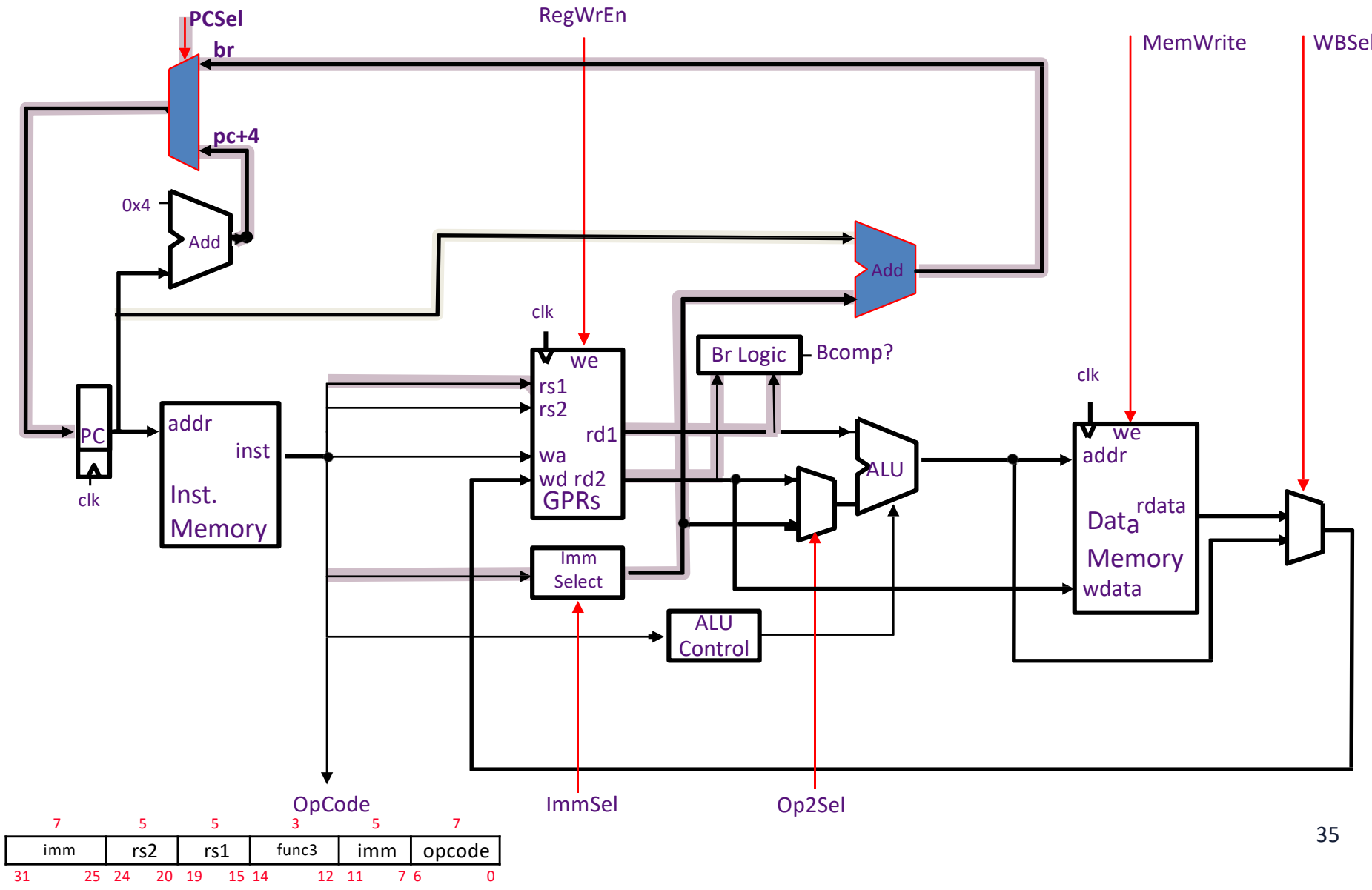
rd ← (rs1) func (rs2)

rd ← (rs1) op immediate

Datapath per istruzioni load/store



Datapath per istruzioni di salto condizionato



Datapath completo (per ora ….)

